Функция превращает вход в результат

Начнём с цены одной позиции: две пары по 7 990 ₽.

Python · запускаемый пример

```
def line_total(price, quantity):
    return price * quantity

result = line_total(7990, 2)
print(result)
```

price = **7990**

quantity = **2**

7990 × 2

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
15980
```

Результат вернулся в переменную `result`.

↳ def объявляет функцию. Вызов запускает её. return возвращает вычисленное значение.

Список хранит позиции, словарь — поля

Одна позиция корзины состоит из ID размера и количества.

Python · запускаемый пример

```
cart = [
    {"product_size_id": 101, "quantity": 2},
    {"product_size_id": 201, "quantity": 1},
]
for item in cart:
    print(item["product_size_id"],
item["quantity"])
```

cart — список из двух словарей

`for` берёт очередную позицию. `item["quantity"]` читает её поле.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
101 2
201 1
```

Каждая строка вывода: ID размера, затем количество.

↳ В памяти это объекты Python. Для передачи по HTTP их преобразуют в JSON.

Аннотация описывает ожидаемый тип

Обычный Python сам не отклоняет аргумент только из-за аннотации.

Python · запускаемый пример

```python
def twice(quantity: int) -> int:
    return quantity * 2

print(twice(2))
print(twice("2"))
```

quantity: int → ожидаем число

→ `int` в записи функции также описывает ожидаемый результат.

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
4
22
```

4 — число.
22 — строка: `"2" * 2` повторяет текст.

↳ Аннотации помогают читать и проверять код. Проверку входа API выполняет отдельный механизм.

Класс описывает объект с данными

Объект хранит своё состояние в атрибутах.

Python · запускаемый пример

```
class Product:
    def __init__(self, name):
        self.name = name

shoe = Product("North Runner")
print(shoe.name)
```

ОБЪЕКТ SHOE В ПАМЯТИ

Product

`name` → North Runner

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
North Runner
```

↳ Обычный класс ещё не создаёт таблицу и не проверяет входные данные.

Импорт соединяет файлы программы

main-файл собирает приложение из небольших частей.

Файлы учебного приложения — Учебный пример

```
sneaker-examples/
├── demo.py
├── schemas.py
├── services.py
└── routers/
    ├── __init__.py
    └── catalog.py
```

Python · фрагмент demo.py

```python
from fastapi import FastAPI
from routers import catalog

app = FastAPI()
app.include_router(catalog.router)
```

`catalog` — импортированный модуль.
`catalog.router` — объект внутри него.

↳ Импорт даёт доступ к определениям. include_router подключает операции к приложению.

Исключение прерывает обычный путь

Недопустимое количество превращаем в явную ошибку.

Python · запускаемый пример

```
def check_quantity(quantity):
    if quantity <= 0:
        raise ValueError("Нужно больше нуля")
    return quantity

try:
    check_quantity(0)
except ValueError as error:
    print(error)
```

raise → поиск обработчика

При quantity = 0 строка `return quantity` уже не выполнится.

✓ ПРОВЕРЕННЫЙ ВЫВОД

Нужно больше нуля

`except ValueError` поймал ошибку и вывел её текст.

↳ Исключение Python и HTTP-ответ — разные вещи. Их связывает обработка ошибки.

Деньги считаем с Decimal

Десятичную цену создаём из строки, сохраняя исходные цифры.

Python · запускаемый пример

```python
from decimal import Decimal

print(0.1 + 0.2)
print(Decimal("0.1") + Decimal("0.2"))
total = Decimal("7990.00") * 2
print(total)
```

✓ ПРОВЕРЕННЫЙ ВЫВОД

```
0.30000000000000004
0.3
15980.00
```

Decimal("7990.00")

Сначала точное значение цены.
Потом умножение на количество.

↳ float хранит двоичное приближение. Для цен используем согласованную десятичную арифметику.

Пройди по коду самостоятельно

Для каждого вызова проверь условие и найди строку return.

Python · запускаемый пример

```
def can_buy(stock, quantity):
    if quantity <= 0:
        return False
    return quantity <= stock

print(can_buy(3, 2))
print(can_buy(3, 4))
print(can_buy(3, 0))
```

Проверенный вывод

```
True
False
False
```

2 ⩽ 3 → True.
4 ⩽ 3 → False.
0 ⩽ 0 → ранний return False.

↳ Сначала подставь значения. Затем следуй только по той ветке, условие которой выполнено.

Кто запускает FastAPI

Uvicorn принимает сетевое соединение и передаёт запрос приложению.



ИЗ ПАПКИ PRESENTATION/SNEAKER-EXAMPLES · ПОСЛЕ УСТАНОВКИ ЗАВИСИМОСТЕЙ

```
python -m uvicorn minimal:app --port 8000
```

↳ minimal — файл minimal.py. app — объект FastAPI внутри этого файла.

Минимальное приложение — семь строк

Этот файл можно запустить и проверить настоящим HTTP-запросом.

Python · запускаемый пример

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok"}
```

1. **Импортировать FastAPI**
   Получить класс из установленной библиотеки.
2. **Создать app**
   Объект приложения хранит маршруты.
3. **Зарегистрировать /health**
   GET по этому пути вызывает health().
4. **Вернуть данные**
   Словарь станет JSON-ответом клиенту.

↳ Проверенный результат GET /health: 200 и {"status": "ok"}.

Метод и путь выбирают обработчик

Декоратор связывает HTTP-операцию с Python-функцией.

GET `/health`

Python · фрагмент minimal.py

```python
@app.get("/health")
def health():
    return {"status": "ok"}
```

Совпали метод и путь

Вызываем `health()`.
Получаем 200.

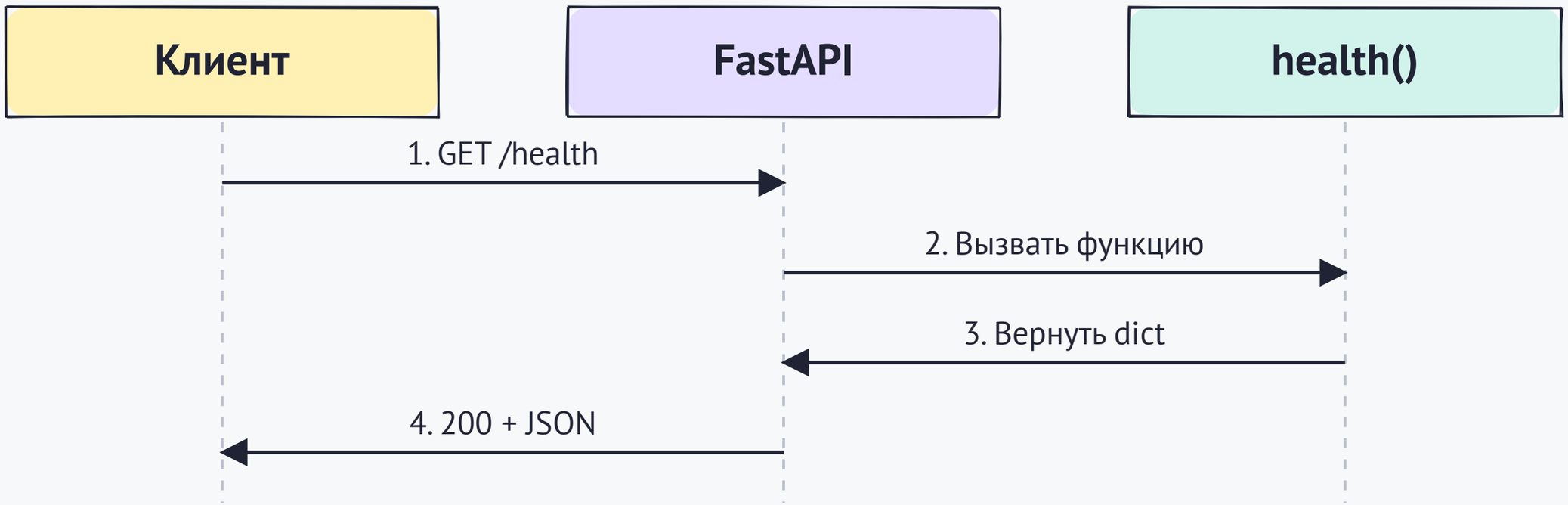
`POST /health` → 405
Путь есть, метод не поддерживается.

`GET /missing` → 404
Такого маршрута в примере нет.

↳ Имя функции — для Python. Внешний адрес задаётся в декораторе.

Проследим один запрос до ответа

Внутри обработчика работаем с Python-данными. Клиент получает HTTP.



↳ Запрос → выбор функции → return → сериализация → ответ.

Проверяем запрос через Swagger

Открой /docs у запущенного учебного приложения.

СХЕМА ИНТЕРФЕЙСА /DOCS

`GET /health`

1. Открыть операцию
2. Нажать **Try it out**
3. Нажать **Execute**

200 `{"status": "ok"}`

1. **Проверить адрес**
   http://127.0.0.1:8000/docs
2. **Прочитать Server response**
   Статус и тело после Execute — фактический ответ.
3. **Отличить пример от ответа**
   Example Value показывает образец схемы. Он не доказывает выполнение запроса.

↳ Swagger отправляет обычный HTTP-запрос. Это наш клиент для практики.

Pydantic проверяет данные на входе

Схема задаёт поля, типы и ограничения запроса.

Python · запускаемый пример

```python
from pydantic import BaseModel, Field

class CartItemCreate(BaseModel):
    product_size_id: int = Field(gt=0)
    quantity: int = Field(gt=0, strict=True)
```

Field(gt=0, strict=True)

Целое число больше нуля.
Строку «2» и boolean не принимаем.

✓ ПРОВЕРЕННЫЙ ВВОД

```
{"product_size_id": 101, "quantity": 2}
```

Такое тело запроса проходит схему.

↳ Форма данных проверена. Наличие размера и остаток ещё предстоит проверить в базе.

Неверный ввод не попадает в обработчик

Переключай количество и сравни проверенные ответы FastAPI.

quantity 0 · ноль

JSON · вход /demo/cart/items

```
{
  "product_size_id": 101,
  "quantity": 0
}
```

422

```
{
  "loc": ["body","quantity"],
  "type": "greater_than",
  "ctx": {"gt":0}
}
```

Тело обработчика: **не вызвано**

quantity должно быть больше 0.

↳ 422 объясняет, где вход не прошёл схему. Вызов тела обработчика проверяем отдельно.

Схема ответа выбирает видимые поля

Внутренний объект может содержать больше данных, чем получает клиент.

Python · схема и сокращённый обработчик

```python
class UserRead(BaseModel):
    id: int
    email: str

@app.get("/demo/user",
         response_model=UserRead)
def example_user():
    return internal_user
```

`id` `email` ~~`hashed_password`~~

✓ ПРОВЕРЕННЫЙ JSON-ОТВЕТ

```
{
  "id": 7,
  "email": "anya@example.com"
}
```

↳ response_model=UserRead фильтрует ответ. Пароль и его хэш клиенту не возвращаем.

На входе и выходе разные данные

Клиент задаёт свойства товара. Сервер добавляет созданный ID.

ЗАПРОС · PRODUCTCREATE

`name` `price`

```
{"name": "North Runner",
 "price": "7990.00"}
```

Название непустое.
Цена — Decimal больше 0.

→

ОТВЕТ · PRODUCTREAD

`id` `name` `price`

```
{"id": 10,
 "name": "North Runner",
 "price": "7990.00"}
```

201 Сервер сформировал ответ.

↳ ProductCreate описывает вход. ProductRead описывает публичный результат.

Depends передаёт готовое значение

FastAPI вызывает зависимость и подставляет её результат в обработчик.

Python · фрагмент demo.py

```python
from fastapi import Depends, Query
def page_size(
    limit: int = Query(10, ge=1, le=50)
):
    return limit

@app.get("/demo/page")
def page(limit: int = Depends(page_size)):
    return {"limit": limit}
```

1 Запрос `?limit=3`

2 `page_size` проверит и вернёт 3

3 `page` получит `limit=3`

↳ Передаём Depends функцию page_size, а не результат её ручного вызова.

Роутер собирает связанные операции

Каталог получает свой файл и общий префикс адресов.

Python · сокращённый routers/catalog.py

```python
from fastapi import APIRouter
from services import list_products

router = APIRouter(prefix="/products")

@router.get("")
def catalog():
    return list_products()
```

`/products` + `""` = `/products`

В demo.py

```
app.include_router(catalog.router)
```

Позже отдельные роутеры:
auth · cart · orders

↳ APIRouter группирует маршруты. include_router делает их частью приложения.

Роутер принимает запрос, сервис решает задачу

Для оформления заказа нам понадобятся разные уровни ответственности.

Роутер

Принять HTTP-запрос.
Вызвать сервис и вернуть ответ.

Псевдокод · будущий POST /orders — Учебный пример

```
пользователь = проверить_доступ()
заказ = сервис.оформить(пользователь)
вернуть_ответ(заказ)
```

Сервис заказа

- ✓ Прочитать свою корзину
- ✓ Проверить остатки
- ✓ Рассчитать сумму
- ✓ Сохранить заказ и списать склад
- ✓ Очистить корзину

Изменения объединит транзакция.

↳ Дальше подключим PostgreSQL и разберём, как эти действия меняют реальные данные.